MONOGRAPH № 02 · 2026-05-08

A Differentiable, GPU-Native Astrodynamics Substrate

A Technical Case for Operational and Strategic Investment

LATEX → PDF · TECHNICAL CASE DOWNLOAD PDF

Reconstruction notice. This page is a summary rebuilt from fragments of the original conversation. The linked PDF is a PDF render of *this reconstruction*, not the original monograph. To replace it with the full source, drop the real `.pdf` into `/pdfs/` using the same filename.

§ Abstract

Operational space domain awareness (SDA) and satellite fleet management both rest on numerical orbit propagators whose architectural choices were locked in two decades ago: hand-derived analytic partial derivatives, CPU-bound integration, and force-model cards configured per-run with no gradient information available downstream. The U.S. Space Force's Special Perturbations Ephemeris (SPEPH) propagator is the operational reference and an excellent example of this architecture; it is also a structural ceiling on what every downstream tool built on top of it can do.

This monograph proposes — and makes the operational and strategic case for — a differentiable, GPU-native astrodynamics substrate that replaces hand-derived partials with automatic differentiation, replaces single-trajectory CPU integration with batched GPU

propagation, and makes the entire stack (propagator, conjunction calculator, scheduler) end-to-end differentiable from action to consequence.

§ Why the current architecture is a ceiling

SPEPH is described in current open documentation as a variable-step Runge-Kutta of the RK 7(8)/RK 8(9) Fehlberg-Dormand-Prince family with high-fidelity force models (EGM-96, IAU-1976/1980 precession-nutation, JPL DE430 third-body, Jacchia-Roberts drag, spherical SRP). The legacy AFSPC code used Gauss-Jackson 8th-order predictor-corrector with an RK starter.

None of that is wrong. What it is, is closed. STMs and partials are hand-derived per force model; the propagator is a black box to anything downstream that wants a gradient with respect to a maneuver, a sensor cue, or a schedule decision. "What ΔV minimizes fuel and preserves the schedule and keeps Pc below threshold" becomes grid search — running the chain forward many times with different candidates — with all of grid search's limitations.

§ The deep-learning analogy

End-to-end differentiability is why deep learning works. A modern neural network is a stack of layers — convolutions, attention, normalizations, classification heads — and the entire stack is differentiable. That is the only reason gradient descent can train it. If any layer were a black box, training would be impossible and the field would not exist.

The proposal, in one sentence: *bring the same architectural property to the operational SDA and fleet-management stack.* The propagator becomes the bottom layer of a learning system; the consequences (schedule revenue, fuel consumption, conjunction probability, customer SLA) become the loss function; the actions (maneuver ΔV's, tasking schedules, sensor cues) become the parameters. The gradient-based optimization that trains transformers becomes the gradient-based optimization that plans maneuvers.

§ What end-to-end differentiability enables

Three concrete capabilities require the chain to be unbroken end-to-end, not just locally differentiable in pieces:

- **Joint optimization across stack layers.** Computing $\partial \mathtt{revenue} / \partial \Delta V$ requires the gradient to flow from the scheduler, through the propagator, through the dynamics, back to the maneuver decision. Any broken link forces the optimization to decompose into separately-optimized pieces and the joint optimum is provably unattainable.
- **Adjoint data assimilation.** Whole-trajectory sensitivity without finite differences.
- **Taylor map diffusion for conjunction Pc.** Three orders of magnitude faster than Monte Carlo on the same ground truth, with gradients included.

§ Demo recommendation

Pick one feature and benchmark it against operational tooling, end-to-end, on a real or representative case. The two highest-impact candidates:

- **(a)** The GA-vs-AD-gradient comparison on a realistic RPO trajectory design — same RHS, same hardware, wall-clock and convergence curves side by side. Most viscerally compelling for a technical audience; the speedup is huge and the comparison is direct.
- **(b)** Taylor Map Diffusion vs. Monte Carlo Pc on a conjunction case from your CDM archive. Same MC ground truth, three orders of magnitude speedup, gradients for free. Most operationally compelling because it touches conjunction analysis directly.

The feature list is what you put in the deck. The benchmark is what closes the room.

§ Status & provenance

Compiled from conversation 2308b2c5 on 2026-05-08, alongside the JAX and Mojo roadmap monographs (Nos. 03 and 04). Originally requested as: *"Summarize this entire conversation as a LaTeX monograph that is self-contained, technical and thorough. Render it as a PDF."*